



ADK Shopping - E-Commerce Multi-Agent System

Welcome to **ADK Shopping**! This project demonstrates how to build a multi-agent e-commerce assistant using **Google's Agent Development Kit (ADK)**. The system uses multiple AI agents working together to provide a complete shopping experience.



Table of Contents

- 1. [What is This Project?](#)
 - 2. [Prerequisites](#)
 - 3. [Agent Workflow Diagram](#)
 - 4. [Understanding the Agent Hierarchy](#)
 - 5. [Detailed Agent Breakdown](#)
 - 6. [How Data Flows Between Agents](#)
 - 7. [Getting Started](#)
-

What is This Project?

This is a **multi-agent shopping assistant** that simulates an e-commerce experience. Instead of one big AI doing everything, we break down the shopping workflow into specialized agents:

- Each agent has a **specific job** (showing products, checkout, order summary)
- Agents **hand off work** to each other like a relay race
- They **share information** through a common state (like a shopping cart)

Think of it like a real store where different employees help you at different stages:

- 🙌 **Greeter** → Welcomes you and understands what you need
 - 🛒 **Sales Associate** → Shows you products and helps you choose
 - 💵 **Cashier** → Collects your shipping details
 - 📄 **Order Desk** → Gives you your final order summary
-

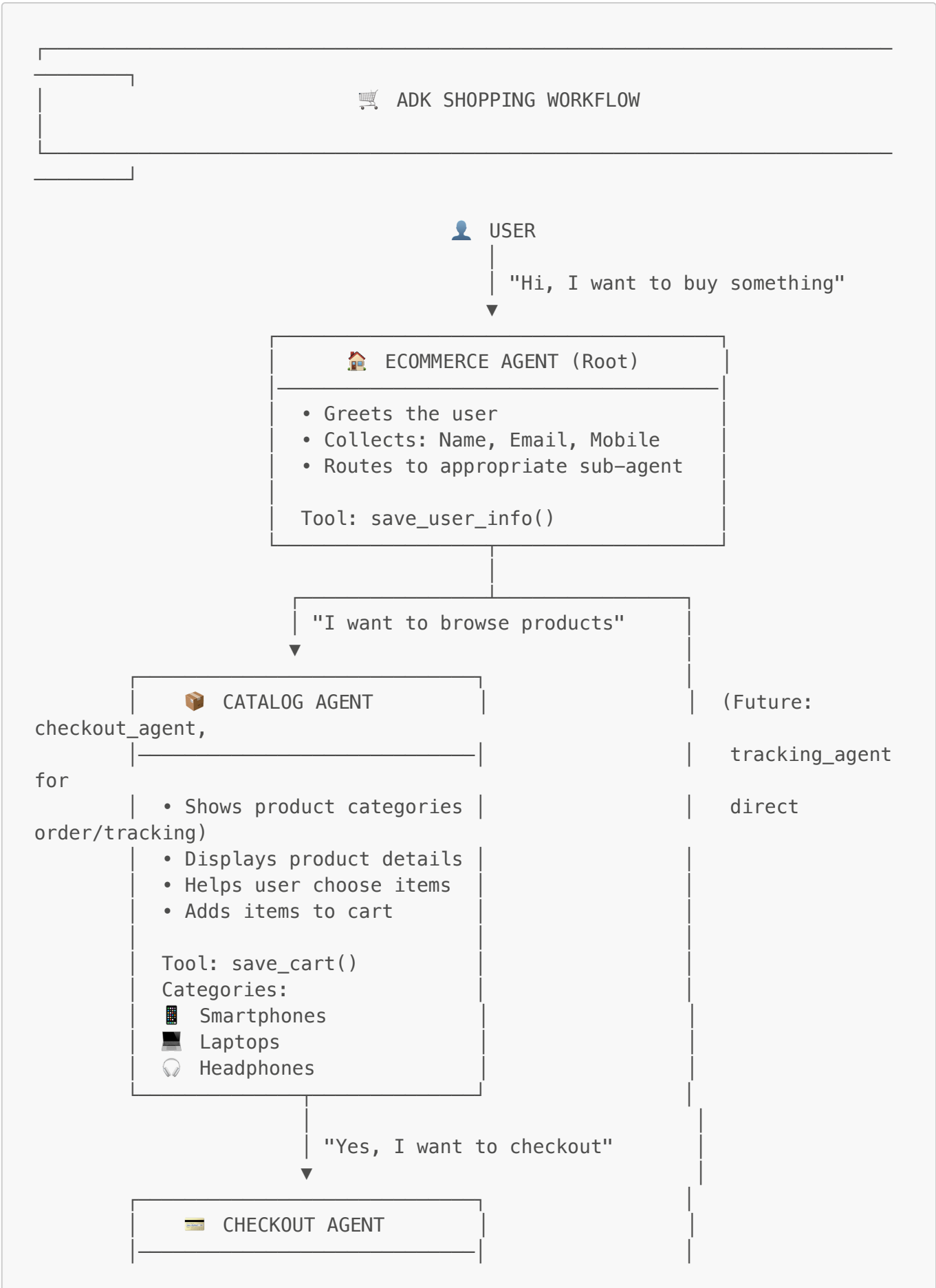
Prerequisites

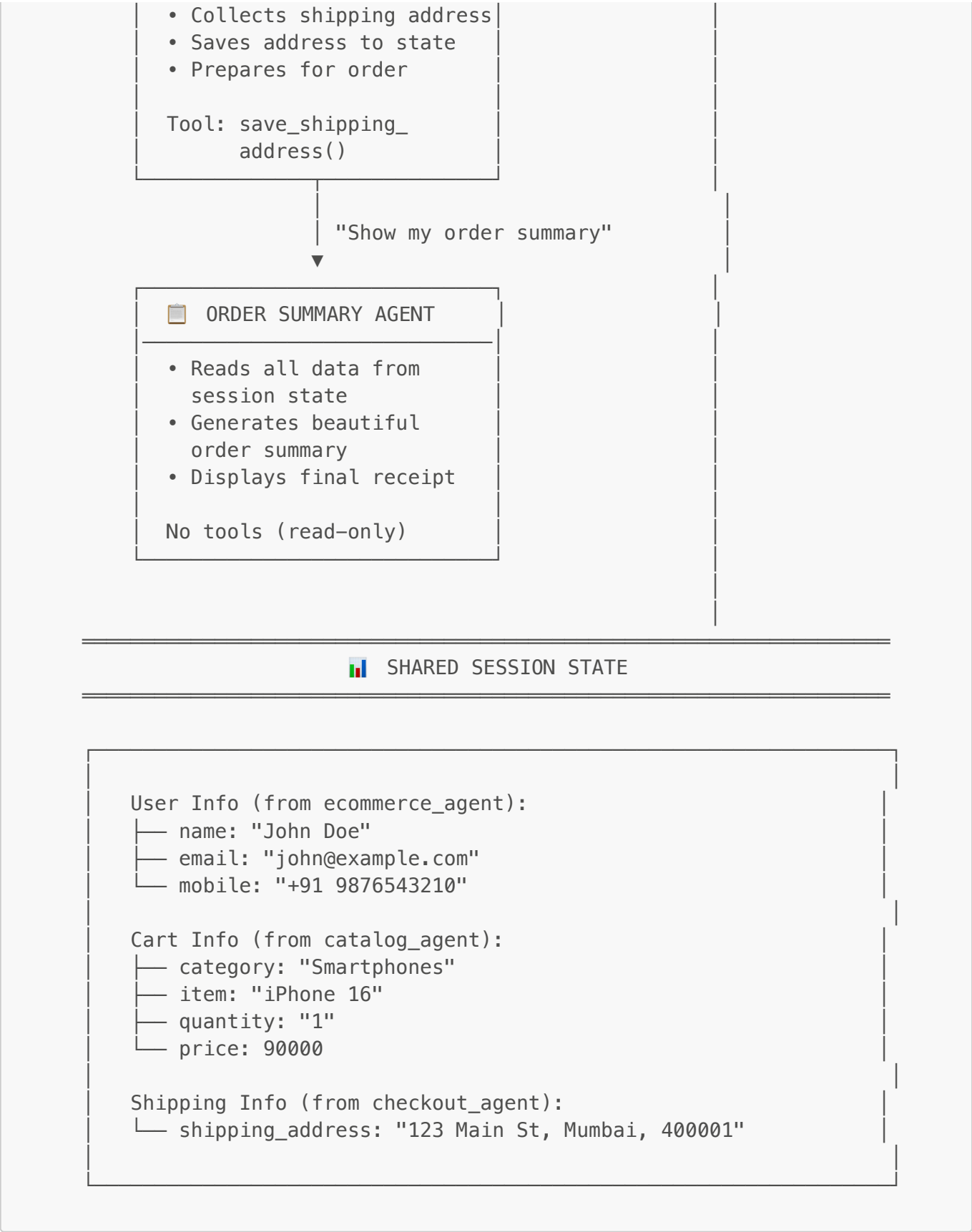
Before diving in, make sure you understand these basic concepts:

Concept	Description
Agent	An AI assistant with a specific role and set of instructions
Sub-agent	An agent that works under another agent (like an employee under a manager)
Tool	A function that an agent can call to perform actions (like saving data)
State	Shared memory where agents store and retrieve information
LLM	Large Language Model - the AI brain powering each agent (Gemini 2.0 Flash in this case)

Agent Workflow Diagram

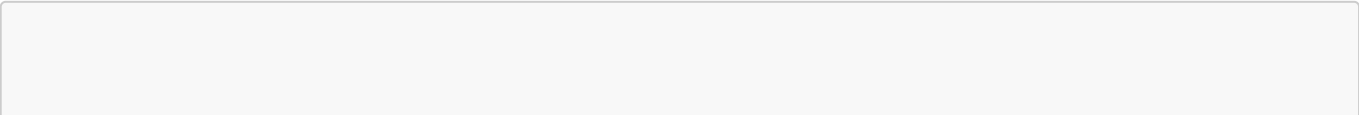
Below is the complete workflow of how agents interact in this e-commerce system:

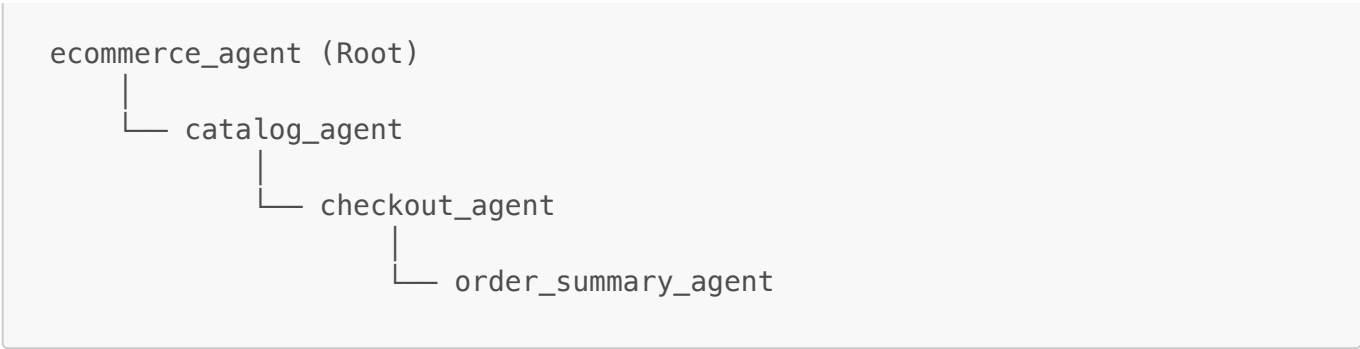




Understanding the Agent Hierarchy

The agents are organized in a **parent-child hierarchy**:





Why This Structure?

Benefit	Explanation
Separation of Concerns	Each agent focuses on ONE job only
Easy Maintenance	Update one agent without breaking others
Natural Flow	Mimics real shopping: Browse → Cart → Checkout → Receipt
Reusability	Agents can be reused in other projects

Detailed Agent Breakdown

1. 🏠 Ecommerce Agent (Root Agent)

File: `ecommerce_agent/agent.py`

Role: The main entry point and orchestrator of the entire shopping experience.

What it does:

- Greets the user when they start chatting
- Collects essential user information:
 - Name
 - Email
 - Mobile number
- Understands user intent (shopping vs tracking orders)
- Routes users to the appropriate sub-agent

Tool:

```
save_user_info(name, email, mobile)
# Saves user details to the shared session state
```

Key Rule: This agent NEVER answers product questions directly. It always delegates to specialized sub-agents.

2. 📦 Catalog Agent

File: `catalog_agent/agent.py`

Role: The product expert that helps users browse and choose products.

What it does:

- Shows available product categories
- Displays product details with prices
- Helps users compare products
- Adds items to the shopping cart

Available Products (Demo Catalog):

Category	Product	Price (₹)
 Smartphones	Pixel 9	70,000
 Smartphones	iPhone 16	90,000
 Smartphones	Galaxy S25	75,000
 Laptops	MacBook Air M3	1,10,000
 Laptops	Dell Inspiron	65,000
 Headphones	Sony WH-1000XM6	30,000
 Headphones	Boat Rockerz	2,000

Tool:

```
save_cart(category, item, quantity, price)
# Saves the selected product to the shopping cart in session state
```

Key Rule: This agent stays in the "catalog domain" only. It cannot place orders or track deliveries.

3. Checkout Agent

File: `checkout_agent/agent.py`

Role: The checkout specialist that collects shipping information.

What it does:

- Asks for the user's shipping address
- Saves the address to session state
- Confirms if user wants to see order summary
- Hands off to the order summary agent

Tool:

```
save_shipping_address(address)
# Saves the shipping address to session state
```

Key Rule: This agent focuses ONLY on collecting shipping details, nothing else.

4. 📄 Order Summary Agent

File: `order_summary_agent/agent.py`

Role: The final step that presents a complete order summary.

What it does:

- Reads ALL information from the session state:
 - User info (name, email, mobile)
 - Cart info (item, quantity, price)
 - Shipping info (address)
- Generates a beautiful, Amazon/Flipkart-style order summary
- Presents the final receipt to the user

Tools: None (this is a read-only agent)

Key Rule: This agent ONLY reads from state. It never invents or guesses information.

How Data Flows Between Agents

The magic happens through the **Session State** - think of it as a shared shopping cart that all agents can read from and write to.

Step-by-Step Data Flow:

```
Step 1: User starts conversation
    └─ ecommerce_agent asks for name, email, mobile
    └─ Saves to state: {name, email, mobile}

Step 2: User wants to browse products
    └─ catalog_agent shows categories and products
    └─ User selects a product
    └─ Saves to state: {category, item, quantity, price}

Step 3: User wants to checkout
    └─ checkout_agent asks for shipping address
    └─ Saves to state: {shipping_address}

Step 4: User wants order summary
    └─ order_summary_agent reads ENTIRE state
    └─ Generates and displays final order summary
```

State Object at the End:

```
session_state = {  
    # From ecommerce_agent  
    "name": "John Doe",  
    "email": "john@example.com",  
    "mobile": "+91 9876543210",  
  
    # From catalog_agent  
    "category": "Smartphones",  
    "item": "iPhone 16",  
    "quantity": "1",  
    "price": 90000,  
  
    # From checkout_agent  
    "shipping_address": "123 Main St, Mumbai, 400001"  
}
```

Getting Started

1. Install Dependencies

```
pip install -r requirements.txt
```

This installs:

- **google-adk** - Google's Agent Development Kit
- **python-dotenv** - For managing environment variables

2. Set Up Environment Variables

Create a **.env** file in the project root:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

3. Run the Agent

```
adk run ecommerce_agent
```

Or use it in your Python code:

```
from ecommerce_agent.agent import root_agent

# The root_agent is now ready to handle conversations
```

Key Takeaways

1. **Multi-agent architecture** breaks complex tasks into manageable pieces
2. **Each agent has a single responsibility** (catalog, checkout, summary)
3. **Agents communicate through shared state** (not direct calls)
4. **Tools are functions** that agents use to perform actions and save data
5. **Sub-agents** are registered under parent agents to enable handoffs

Further Learning

- [Google ADK Documentation](#)
- [Gemini API Documentation](#)
- Learn about agent orchestration patterns
- Explore adding more features like:
 - Order tracking agent
 - Payment processing agent
 - Customer support agent

Happy Coding! 

Built with  using Google ADK and Gemini 2.0 Flash